Software Development Fundamentals — Tugas Praktik Sesi 10

Tujuan Tugas

Mahasiswa mampu menulis unit test yang benar-benar menjaga perilaku kode, menggunakan fitur PyTest atau JUnit 5 secara tepat, dan membuktikan daya deteksi test melalui pengujian mutasi sederhana.

Berkas yang Disediakan

Modul yang harus diuji tersedia pada berkas `pricing.py` dengan isi sebagai berikut.

```
TAX_RATE = 0.11
MEMBER_DISCOUNT_RATE = 0.10


def calculate_subtotal(unit_price, quantity):
    if unit_price < 0:
        raise ValueError("unit_price must not be negative")
    if quantity <= 0:
        raise ValueError("quantity must be positive")
    return unit_price * quantity


def apply_member_discount(subtotal, is_member):
    if is_member:
        return subtotal * (1 - MEMBER_DISCOUNT_RATE)
    return subtotal


def add_tax(amount):
    return amount * (1 + TAX_RATE)


def calculate_total(unit_price, quantity, is_member=False):
    subtotal = calculate_subtotal(unit_price, quantity)
    discounted = apply_member_discount(subtotal, is_member)
    return add_tax(discounted)


def write_receipt(path, lines):
    with open(path, "w", encoding="utf-8") as handle:
        handle.write("\n".join(lines) + "\n")
```

Aturan bisnis yang harus dipatuhi modul tersebut:

1. Subtotal adalah harga satuan dikali kuantitas.
2. Harga satuan negatif ditolak dengan `ValueError`.
3. Kuantitas nol atau negatif ditolak dengan `ValueError`.
4. Anggota menerima diskon sepuluh persen, non-anggota membayar penuh.
5. Pajak sebelas persen ditambahkan pada nilai setelah diskon.
6. Struk ditulis ke berkas dengan setiap baris diakhiri baris baru, termasuk baris terakhir.

Instruksi

1. **Struktur.** Tempatkan test pada direktori `tests` dengan nama berkas yang memenuhi aturan penemuan otomatis PyTest, sehingga perintah `pytest` dapat menemukan seluruh test tanpa argumen tambahan.
2. **Cakupan perilaku.** Tulis minimal **dua belas** fungsi test yang mencakup seluruh aturan bisnis pada poin 1 sampai 6, termasuk kasus tepi dan jalur kesalahan.
3. **Penggunaan fitur.** Wajib menggunakan:
   - `pytest.mark.parametrize` pada minimal satu test dengan sekurang-kurangnya tiga kombinasi input;

    - `pytest.raises` pada minimal dua test untuk memverifikasi pengecualian;
    - `pytest.approx` pada minimal satu test yang melibatkan perhitungan bilangan floating point;
    - fixture `tmp_path` untuk menguji fungsi penulisan struk.
4. **Kualitas assertion.** Setiap test harus memeriksa nilai atau perilaku yang konkret. Assertion yang selalu bernilai benar, misalnya `assert result is not None` atau `assert x < 100 or x >= 100`, tidak diterima.
5. **Pengujian mutasi.** Buat tiga kerusakan buatan pada salinan `pricing.py` secara berurutan, kemudian catat test mana yang gagal untuk setiap kerusakan. Sajikan hasilnya dalam tabel dengan format berikut.

Kerusakan buatan	Test yang gagal	Kesimpulan daya deteksi

Kerusakan yang disarankan: menghapus penerapan diskon, mengubah tarif pajak, dan menghapus salah satu validasi masukan.

6. **Jalankan pengujian** dan lampirkan keluaran perintah `pytest -v` sebagai bukti bahwa seluruh test lulus pada kode yang benar.

Bagian untuk Mahasiswa yang Memilih JUnit 5

Jika Anda memilih Java, penuhi ketentuan berikut sebagai pengganti fitur PyTest di atas.

1. Gunakan anotasi `Test` dengan assertion dari JUnit 5.
2. Gunakan `assertThrows` pada minimal dua test.
3. Gunakan `ParameterizedTest` dengan `ValueSource` atau `CsvSource` untuk minimal tiga kombinasi input.
4. Gunakan `BeforeEach` untuk penyiapan yang dipakai lebih dari satu test.
5. Gunakan direktori sementara yang disediakan JUnit, misalnya `TempDir`, untuk pengujian penulisan berkas.

Pertanyaan Analisis

1. Jelaskan mengapa perbandingan hasil perhitungan bilangan floating point sebaiknya memakai `pytest.approx` alih-alih perbandingan kesetaraan langsung.
2. Jelaskan mengapa test yang memanggil berkas nyata di dalam direktori proyek lebih berisiko dibandingkan penggunaan `tmp_path`.
3. Jelaskan satu test pada hasil pekerjaan Anda yang menurut Anda paling lemah, beserta alasan dan usulan perbaikannya.
4. Jelaskan bagaimana hasil pengujian mutasi mengubah cara Anda menilai kualitas sebuah test.
5. Jelaskan padanan antara `pytest.raises`, `pytest.mark.parametrize`, dan fixture pada PyTest dengan fitur yang setara pada JUnit 5.
6. Jelaskan mengapa pernyataan "total anggota selalu lebih besar daripada subtotal" merupakan asumsi yang keliru pada modul ini, dengan menunjukkan perhitungannya.

Artefak yang Dikumpulkan

- Berkas `pricing.py` yang tidak diubah sebagai acuan.
- Direktori `tests` berisi berkas test yang Anda tulis.
- Keluaran `pytest -v` pada kode yang benar.
- Tabel hasil pengujian mutasi untuk tiga kerusakan.
- Jawaban Pertanyaan Analisis.